

SMARTCAR Project

(Technical document)

Contents

1. Introduction	4
2. Overview	4
2.1 Purpose	4
2.2 Scope	4
2.3 Audience	5
3. Architecture Overview.....	5
3.1 High-Level Architecture.....	5
3.2 Component Diagram	6
4. Detailed Architecture	6
4.1 Architecture diagram	6
4.2 Web Client	6
<i>User Interface</i>	7
4.2 Backend Server	7
API Gateway.....	7
Application Logic.....	7
Data Access Layer	7
MQTT Integration Layer	7
4.3 Sensor Simulator	8
5. Database	8
5.1 ERD	9
5.2 User table	9
5.3 Driver Table	9
5.4 Admin Table	10
5.5 Car Table	10
5.6 Telemetry Table	10
5.7 Alert Table	10
6. Data Flow	11
6.1 Sensor Data transmission via Simulator	11
6.2 Reception and Processing by Backend Server	11
6.3 Database Logging and Alert Generation.....	11

6.4 User Actions via Web Interface	11
6.5 Backend Response and UI Update	11
7. Security Considerations	11
7.1 Authentication and Authorization	12
7.3 Data Governance and Monitoring.....	12
8.Conclusion	12

1. Introduction

This document presents the high-level architecture of the SmartCar monitoring platform, a web-based system developed to oversee vehicle telemetry and real-time. It outlines the core components of the application, including the React-based user interface, Spring Boot backend services, MySQL database, and an MQTT-driven telemetry simulator, with a focus on how telemetry data is transmitted, processed, and visualized. The architecture supports two primary user roles “Driver” and “Admin” each with dedicated dashboards for interacting with vehicle data and system functions. Telemetry inputs such as speed, temperature, fuel level, and location are generated at regular intervals and routed through an MQTT broker to the backend, where they are stored and analysed. This document emphasizes system design, data flow, and integration strategy, while excluding detailed implementation, UI design elements, and module-level code.

2. Overview

2.1 Purpose

The purpose of this document is to describe the architectural framework of the SmartCar monitoring application, highlighting its core components, system interactions, and the technologies employed to deliver its functionality. It provides an overview of how the application integrates MQTT-based telemetry simulation with backend services to collect, process, and present vehicle data in real-time. The document explains how the system supports role-based access for drivers and administrators, enabling personalized dashboards for vehicle tracking and management. By detailing the structural layers and essential modules, this document offers insight into backend operations and their connection to the React-based frontend.

2.2 Scope

This document outlines the overarching architecture of the SmartCar monitoring application, detailing its principal components, system workflows, and backend operations, with a particular focus on the integration of MQTT-based telemetry simulation. It highlights how various technologies—React, Spring Boot, MySQL, and MQTT—work together to enable real-time vehicle tracking and driver management. The scope also includes deployment considerations and scalability strategies to ensure the system can reliably handle continuous data streams and support multiple users simultaneously. The application is designed to manage telemetry data, generate alerts, and provide role-specific dashboards for both drivers and administrators. This

document does not delve into detailed implementation aspects, frontend design elements, or the internal code structure of individual modules

2.3 Audience

This document is intended for developers, solution architects, project managers, and other stakeholders involved in building, deploying, and maintaining the SmartCar monitoring system. It provides a comprehensive understanding of the system architecture to support collaboration across technical teams. By detailing the major components and data flow, it enables informed decision-making for future enhancements, integrations, and scalability planning. This document serves as a foundational reference for understanding how telemetry data is processed and how user roles interact with the system.

3. Architecture Overview

3.1 High-Level Architecture

The high-level architecture of the SmartCar monitoring system is composed of the following key components:

Web Client: The frontend application built using React, providing an intuitive interface for both drivers and administrators. It enables users to log in, view assigned vehicle details, monitor real-time telemetry data, and manage system interactions based on their roles.

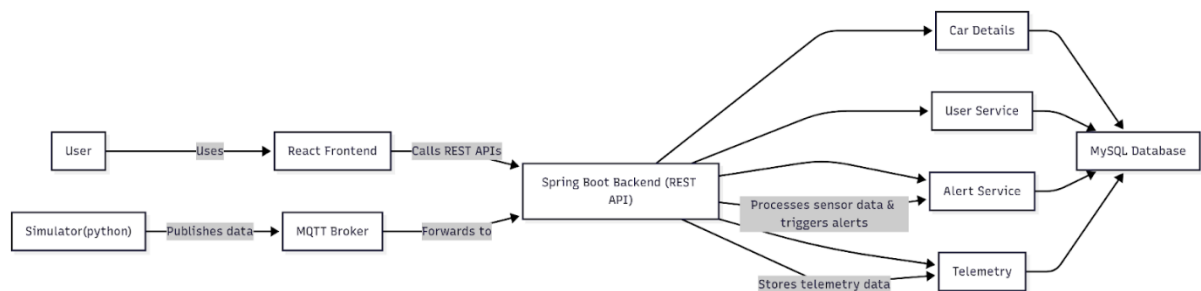
Backend Service: A Spring Boot-based server that handles client requests, enforces business logic, manages user roles, and communicates with the MQTT broker to process incoming telemetry data. It also exposes RESTful APIs for frontend communication and interacts with the database for data persistence.

MQTT Simulator & Broker: A telemetry simulator that publishes vehicle data—such as temperature, speed, fuel level, and location at regular intervals (every 5 seconds). The MQTT broker facilitates reliable message delivery to the backend, ensuring real-time data ingestion and alert generation.

Database (MySQL): A structured relational database used to store user profiles, vehicle assignments, telemetry records, and alerts. It supports efficient querying, data integrity, and secure storage of historical and live data.

This architecture enables a scalable and responsive environment for real-time vehicle monitoring and management, leveraging MQTT for telemetry simulation and REST APIs for seamless frontend-backend communication.

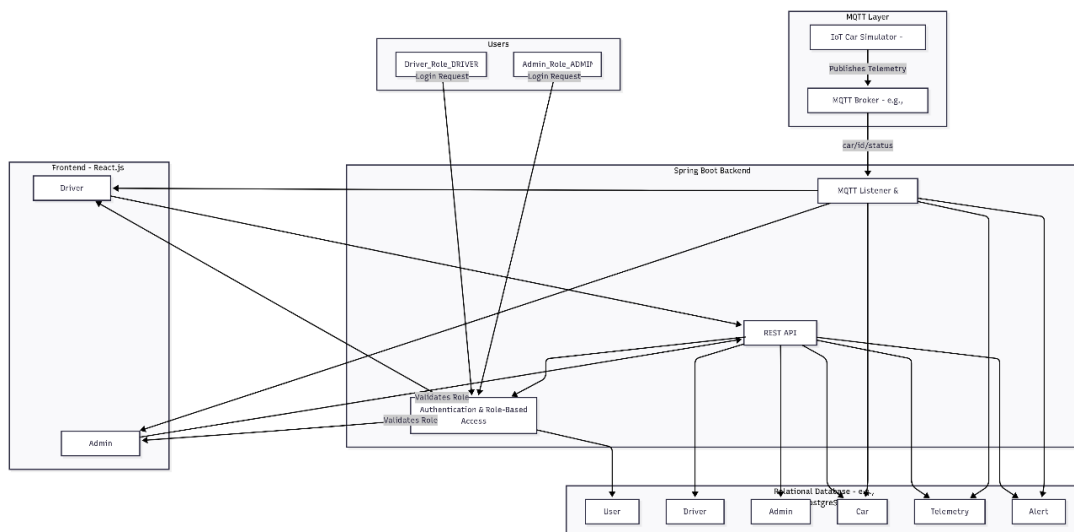
3.2 Component Diagram



This component diagram outlines a modular system architecture integrating All the modules The React UI interacts with the backend via REST APIs. Sensor data is generated by the simulator and sent through an MQTT broker to the backend, which processes telemetry, triggers alerts, and stores data in a MySQL database. The backend is contains of distinct services such Car Details, User Service, Alert Service, and Telemetry, each handling specific business logic.

4. Detailed Architecture

4.1 Architecture diagram



4.2 Web Client

The User Client, built using React, serves as the primary interface for both drivers and administrators. It is designed to deliver a clean, responsive experience and provide real-time access to car performance, alerts.

User Interface

Developed using React, the interface adapts dynamically based on user roles. Drivers are presented with dashboards showing real-time telemetry such as speed, fuel level, and engine temperature for their assigned vehicle. While Admin users have broader access, allowing them to monitor all registered vehicles, manage users, and view alerts system wide. Features like login, authentication, and access control ensure each user interacts only with their permitted data.

4.2 Backend Server

The Backend Server built using Java Spring Boot is the core of the application. It manages business logic, processes telemetry data, enforces alert rules, and ensures secure data access. The backend is organized into several modular components that work together to handle incoming client requests, interact with the database, and respond to real-time sensor input via MQTT.

API Gateway

The API Gateway acts as the central access point for all frontend requests. It handles routing of HTTP requests to the appropriate internal services. It also plays a crucial role in maintaining scalability by enabling throttling, logging, and basic load control mechanisms. Additionally, the API gateway implements essential security features, such as authentication and authorization, Role-based access control and Role-based access control

Application Logic

The Application Logic layer contains modular service classes that execute business logic for core system functionality. These services handle operations such as vehicle management, sensor data processing, alert monitoring, and user administration. By encapsulating distinct responsibilities, they ensure workflow consistency, maintain secure session handling, and uphold application stability across diverse request types.

Data Access Layer

The Data Access Layer (DAL) acts as a bridge between the business logic and the MySQL database, using Spring Data JPA to handle key database operations. It handles transactional updates to maintain data accuracy and consistency.

MQTT Integration Layer

The MQTT Integration Layer supports real-time vehicle monitoring by using a backend subscriber module that connects to a messaging broker. This module listens for telemetry data coming from car sensors. When a new message arrives, the Telemetry Service stores the data in the database, and the Alert Service instantly checks it against predefined thresholds. If necessary, alerts are generated and flagged for display on the

frontend dashboard. This setup ensures users receive quick updates about vehicle performance and health.

4.3 Sensor Simulator

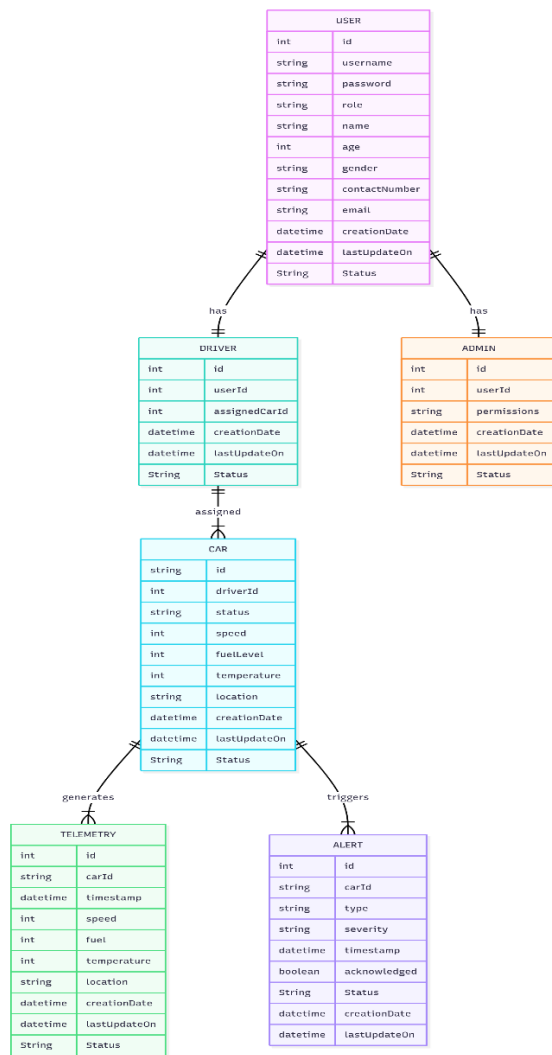
The Sensor Simulator plays an important role in mimicking how smart cars behave by generating synthetic telemetry data. It's used during development, testing, and demos to simulate the output from real vehicle sensors in a consistent and controlled way. The simulator produces regular data readings for parameters like speed, fuel level, engine temperature, and location. Each data point is structured as a JSON message and tagged with metadata like timestamp and car ID to simulate individual vehicle streams.

MQTT Publisher is used to mimic real-time communication, the simulator connects to an MQTT broker and continuously publishes telemetry messages to a defined topic path. These messages are instantly received by the backend's MQTT subscriber module, which stores the data in a telemetry database and evaluates it against threshold rules to check for alerts. If a reading triggers an alert condition, appropriate warnings are created.

5. Database

The Database component, built on MySQL, serves as the central hub for storing all system data, offering secure and efficient access to the database

5.1 ERD



5.2 User table

The USER table serves as foundational entity for managing users in system, it has primary details of both Driver and Admin. Each user is uniquely identified by UserId (PK). The table contains essential personal and account related information , it includes username & password for authentication, role to distinguish between different user such as Driver or Admin. Additional fields like name, age, gender, contactNumber, and email provide demographic and communication details. The creationDate and lastUpdateOn fields are used to track the lifecycle of each user record, ensuring transparency in data changes. The Status field indicates the current state (e.g., Active, Inactive). of the user account. This table has one-to-one relationship to the DRIVER and ADMIN tables, enabling role-specific extensions

5.3 Driver Table

The DRIVER table manages all drivers and maintains key relationships within the system. Each driver is uniquely identified by DriverId(PK) and linked to a user profile

through the `userId` field, forming a one-to-one relationship with the `USER` table. The `assignedCarId` connects the driver to a vehicle, establishing a one-to-many relationship with the `CAR` table, where one driver may be assigned multiple cars over time. Timestamps like `creationDate`, `lastUpdateOn` track the changes in record, and the `Status` field reflects the driver's current state (e.g. Active, Suspended , Inactive)

5.4 Admin Table

The `ADMIN` table stores role-specific data for users with administrative privileges. Each admin is uniquely identified by `adminId` (PK) and linked to a user account through `userId`, forming a one-to-one relationship with the `USER` table. The `permissions` field defines the scope of access or control the admin has within the system. Lifecycle tracking is handled via `creationDate` and `lastUpdateOn`, while the `Status` field indicates the admin's current state (e.g., Active, Revoked). This table extends the core user profile with administrative capabilities, ensuring secure and structured management of system operations.

5.5 Car Table

The `CAR` table captures detailed information about cars in the system. Each car is uniquely identified by the `carId`(PK) field and linked to a driver via `driverId`, forms a many-to-one relationship with the `DRIVER` table. Operational metrics such as speed, `fuelLevel`, temperature, and location are stored to monitor vehicle performance. The `status` field reflects the car's current condition (e.g., Available, In Use, Maintenance), while `creationDate` and `lastUpdateOn` track its lifecycle. The `CAR` table also has one-to-many relationships with both the `TELEMETRY` and `ALERT` tables, enabling real-time data generation and alert triggering for each car.

5.6 Telemetry Table

The `TELEMETRY` table records real-time operational data for all cars. Each entry is uniquely identified by `TelemetryId` (PK) and forms many-to-one relationship with `CAR` table via `carId` for a specific car. It captures dynamic metrics such as speed, fuel, temperature, and location, along with a timestamp to mark when the data was collected. The `creationDate` and `lastUpdateOn` fields support auditability, while the `Status` field indicates the validity of the telemetry record (e.g., Active, Archived). This table enables continuous monitoring of vehicle performance and supports analytics, diagnostics, and alert generation.

5.7 Alert Table

The `ALERT` table is designed to capture and manage critical notifications triggered by vehicle conditions, based on data recorded in the `TELEMETRY` table.. Each alert is uniquely identified by `alertId`(PK) and associated with a specific car via `carId`, forming a many-to-one relationship with the `CAR` table. It includes fields like `type` (e.g., Speeding,

Low Fuel), severity (e.g., High, Medium, Low), and timestamp to indicate when the alert occurred. The acknowledged field (boolean) tracks whether the alert has been reviewed or addressed. Lifecycle fields creationDate and lastUpdateOn provide audit trails, while the Status field reflects the alert's current state (e.g., Active, Resolved). This table supports proactive vehicle management and safety monitoring.

6. Data Flow

The data flow outlines how information travels through the system from sensor data generation and user interactions on the web interface to backend processing and response delivery

6.1 Sensor Data transmission via Simulator

The system begins with a Python-based simulator that emulates real-time car telemetry by publishing sensor data such as speed, fuel level, temperature, and location to an MQTT broker.

6.2 Reception and Processing by Backend Server

The backend server, subscribed to MQTT topics, receives incoming telemetry data. It validates the input and checks it against predefined threshold rules to determine whether alerts need to be raised.

6.3 Database Logging and Alert Generation

Valid telemetry data is stored in the MySQL database for historical tracking. If a data point exceeds safe limits, the system creates an alert entry, also recorded in the database.

6.4 User Actions via Web Interface

Drivers and admin users interact with the React-based web application to view assigned vehicles, check usage history, or monitor alerts. Their actions trigger REST API calls to the backend.

6.5 Backend Response and UI Update

The backend processes these user requests, retrieves relevant data, and returns structured responses. The frontend then updates the user interface in real-time, displaying live car status, alerts, or system feedback accordingly.

7. Security Considerations

Due to real-time nature of vehicle telemetry and user-specific access to sensitive data maintaining a secure system architecture is essential.

7.1 Authentication and Authorization

Authentication Mechanism: Implements JWT-based token authentication to ensure every request is verified and linked to an active user session.

Role-Based Access: Enforces permissions based on user roles (e.g., admin or driver), controlling access to APIs and features like car management, usage reports, and alert handling.

7.3 Data Governance and Monitoring

Limited Privacy Controls: The system currently offers basic user account management features, including account deletion and data review.

8.Conclusion

This document presents a high-level overview of the architecture for the Smart Car Monitoring System, which enables real-time vehicle tracking, alert generation, and user-specific data access through a robust web interface. Built using a modular, scalable approach with technologies like React, Spring Boot, MQTT, and MySQL, the system ensures smooth integration between simulated sensors, backend logic, and user-facing dashboards.

Designed with performance, maintainability, and security in focus, the architecture supports role-based access for both drivers and admins while providing reliable telemetry monitoring. Future enhancements could include advanced data analytics for usage prediction, integration with GPS-based routing, and support for additional sensor types to broaden the system's capabilities across a wider range of smart mobility scenarios.